Machine Coding: OTP Input

- **Problem:** Build an OTP (one-time password) input: N separate digit boxes with auto-focus, auto-advance, Backspace, arrow-key navigation, paste of the whole code, SMS autofill, a red border for missing digits and auto-submit once the code is complete.
- **Tech stack:** React 19 (Vite, JavaScript), plain CSS
- **Time limit:** 60 minutes

PART A: Thought process

The same steps work for any machine coding task:

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases:** every action and its expected result. This is your checklist.
3. **Design the architecture:** components, helpers, services and who owns what.
4. **Design the state:** the smallest set of state; derive the rest.
5. **Design data and contracts:** example data, API shape, component props.
6. **Trace the data flow:** one interaction end to end before writing code.
7. **Find the hard parts:** the 2-4 things that break naive versions.
8. **Plan the build order:** smallest working version first, then one layer at a time.
9. **Cover states and edge cases:** initial, typing, complete, verifying, success, error, plus problem-specific ones.
10. **Fix the file structure.**
11. **Write the code.**
12. **Wrap up:** what is handled, trade-offs, what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Use cases, architecture, state, contracts	7 min
Build: N boxes, state array, typing + auto-advance	10 min
Build: Backspace, arrows, focus helper	8 min
Build: paste + SMS autofill	8 min
Build: validation (red border), auto-submit, mock verify	10 min
Edge cases, accessibility, mobile, walkthrough	12 min

Think out loud. Say what you are deciding and why ("I'll store an array of digits, not a string, because each box needs its own slot, and `join('')` gives me the code for free"). The interviewer grades your reasoning as much as the result, and talking lets them steer you early.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render `length` boxes (default 6), one digit each. `length` is a prop.
- Focus the first box on mount.
- Accept digits only; ignore letters, spaces and symbols.
- After a digit is entered, move focus to the next box.
- **Backspace:** clears the current box; on an empty box, moves back and clears the previous one.
- **ArrowLeft / ArrowRight:** move focus between boxes.
- **Paste:** pasting a code (e.g. `123456` or `123-456`) fills the boxes from the focused box and keeps only digits.
- **SMS autofill:** support the browser's one-time-code autofill (`autocomplete="one-time-code"`).
- **Validation:** submitting an incomplete code shows a red border on every empty box and a message.
- **Auto-submit:** once all boxes are filled, submit after a short delay (2 s); editing during that delay cancels it.
- Verify the code with an API (mocked); show verifying, success and error states. On error, clear the boxes and focus the first one.

Non-functional requirements

- **Performance:** trivial data (6 strings); no extra renders or timers left running.
- **Error handling:** wrong code and network failure both show a message and let the user retry; no double submits.
- **Accessibility:** every box has an `aria-label` ("Digit 3 of 6"), the group has a label, errors use `role="alert"`, visible focus ring, full keyboard support, numeric keypad on mobile (`inputMode="numeric"`).
- **Reusability:** `OtpContainer` takes `length`, `autoSubmitDelay`, `disabled`, `onSubmit`; it knows nothing about the API.
- **Extensibility:** masking (password dots), alphanumeric codes or a resend timer can be added without touching the core logic.

Clarifying questions to ask the interviewer

1. Digits only, or alphanumeric codes too?
2. Should paste work in any box or only in the first? Fill from the focused box or always from box 1?
3. Auto-submit, a Verify button, or both? How long should the delay be?
4. What happens after a wrong code: clear all boxes, or keep them and highlight?
5. Do we need a "Resend code" button with a countdown?
6. Should the digits be masked like a password?

Assumptions we will build with

- Digits only, length 6 by default (configurable).
- Paste works in any box and fills from that box onward (pasting in box 1 fills the whole code).
- Both auto-submit (2 s after the last digit) and a Verify button.

- Wrong code: clear all boxes, show the error, focus box 1.
- No resend timer and no masking (mentioned as extensions).
- The API is mocked: `123456` is the valid code.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	6 empty boxes, box 1 focused
Type `5` in an empty box	Box shows `5`, focus moves to the next box
Type `5` in the last box	Box shows `5`, focus stays (no box 7)
Type a letter / space / `-`	Ignored; box keeps its value, focus does not move
Type `7` into a box that already holds `5`	Box becomes `7` (replace, not append), focus moves on
Backspace on a filled box	Box cleared, focus stays
Backspace on an empty box	Previous box cleared and focused
Backspace on empty box 1	Nothing happens
ArrowLeft / ArrowRight	Focus moves one box; stops at the ends
Click / focus a box	Its digit is selected so the next key replaces it
Paste `123456` in box 1	All boxes filled, focus on the last box
Paste `12 34-56x`	Non-digits stripped, `123456` filled
Paste `99` in box 5 of 6	Box 5 = 9, box 6 = 9; extra digits dropped
Paste `abc`	Nothing changes
SMS autofill puts `123456` in box 1	Treated like a paste: all boxes filled
Click Verify with 4 of 6 digits	Empty boxes get a red border, "Enter all 6 digits", focus first empty box
Fill the last empty box	Red borders disappear; "Submitting automatically..." hint; submits after 2 s
Edit a digit during the 2 s delay	Timer cancelled; restarts when the code is complete again
Click Verify during the 2 s delay	Submits once (timer cancelled, no double request)
Verifying	Boxes and button disabled, button says "Verifying..."
Wrong code	Boxes cleared, box 1 focused, error message shown
Correct code	Success message replaces the form

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`App`	Page shell; owns verification status, calls the API, resets the OTP after a wrong code
`OtpContainer`	Owns the digit array, refs and focus; handles typing, Backspace, arrows, paste, validation, auto-submit
`OtpInput`	One presentational box: renders an `<input>` with the right attributes and forwards events
`utils/otp.js`	Pure helpers: `emptyOtp`, `newDigits` (what did the user just type?), `fillFrom` (write digits from an index)
`api/verifyOtp.js`	Mock API: resolves for `123456`, rejects otherwise

```
App  (status, message, attempt)
 |-- OtpContainer  key={attempt}  (otp[], showErrors, inputRefs)
 |     |-- OtpInput x length   (value, invalid, ref)
 |     |-- error / hint text, Verify button
 |     `-- uses utils/otp.js
 `-- verifyOtp()  (mock API)
```

Start with everything in `OtpContainer`; pull out `OtpInput` once the `<input>` props grow, and pull the pure logic into `utils/otp.js` so it can be tested without a browser.

---

4. Design the state / data model

State or field	Owner	Purpose
`otp: string[]`	OtpContainer	One slot per box, `''` when empty
`showErrors: boolean`	OtpContainer	Turned on after a submit with missing digits; drives red borders
`inputRefs: (HTMLInputElement)[]` (ref)	OtpContainer	Focus a box by index; not state because it never affects rendering
`status: 'idle' | 'verifying' | 'success' | 'error'`	App	Verification lifecycle
`message: string`	App	Success / error text
`attempt: number`	App	Used as `key` on `OtpContainer`; bumping it remounts and clears the boxes

```
// OtpContainer
const [otp, setOtp] = useState(() => emptyOtp(length))
const [showErrors, setShowErrors] = useState(false)
const inputRefs = useRef([])

// App
const [status, setStatus] = useState('idle')
const [message, setMessage] = useState('')
const [attempt, setAttempt] = useState(0)
```

Senior note: derive, do not store.

- `code = otp.join('')` and `isComplete = code.length === length` are derived every render. Storing them would mean keeping three values in sync.
- `invalid` for a box is `showErrors && !digit`: as soon as the user fills a red box, it turns normal without any extra code.
- "Which box is focused" is not state. The DOM already knows (`document.activeElement`), and storing it would cause a re-render on every focus change.
- The digits live in `OtpContainer`, not in `App`: `App` only needs the final code, which arrives through `onSubmit(code)`. If a parent ever needs the live value, add an `onChange(code)` prop or lift the state then.

---

5. Design the data structures and contracts

Example data

```
emptyOtp(6)                   // ['', '', '', '', '', '']
['1', '2', '', '4', '', '']   // partially filled
['1', '2', '3', '4', '5', '6'] // complete -> code '123456'
```

An array of strings (not a string, not numbers) because:

- each box needs its own slot, including empty slots in the middle;
- a string would lose position information for gaps (`'12 4'` is fragile);
- numbers would turn `'0'` into `0`, which is falsy and breaks `every(Boolean)`-style checks, and leading zeros matter in OTPs.

Keys: the boxes are a fixed-size positional list that never reorders, so `key={index}` is correct here.

Component contracts

```
<OtpContainer
  length={6}               // number of boxes
  autoSubmitDelay={2000}   // ms after the last digit
  disabled={false}         // true while verifying
  onSubmit={(code) => {}}  // called once with '123456'
/>

<OtpInput ref index length value invalid disabled onChange onKeyDown onPaste />
```

Helper contracts (pure, testable in Node)

```
newDigits(raw, oldDigit) // -> string | null
newDigits('7', '')       // '7'
newDigits('57', '5')     // '7'   (typed next to the old digit)
newDigits('a', '')       // null  (ignore)
newDigits('123456', '')  // '123456' (autofill)

fillFrom(otp, index, digits) // -> { otp: string[], focus: number }
fillFrom(['','','','','',''], 0, '12') // { otp: ['1','2','','','',''], focus: 2 }
```

API contract (mocked)

```
verifyOtp(code: string) -> Promise
  resolves { ok: true }                when code === '123456' (after 800 ms)
  rejects  Error('Invalid code. ...')   otherwise
```

Real-world equivalent: `POST /api/otp/verify { "phone": "+91...", "code": "123456" }` -> `200 { "verified": true }` or `400 { "error": "INVALID_CODE", "attemptsLeft": 2 }`.

---

6. Trace the data flow

Interaction: the user types the last digit

```
User types "6" in box 6 (boxes 1-5 hold 1..5)
   |
   v
<input> fires change  -> OtpInput forwards onChange
   |
   v
handleChange(5, e)
   newDigits('6', '') -> '6'
   fillFrom(otp, 5, '6') -> { otp: ['1'..'6'], focus: 5 }
   setOtp(next); focusBox(5)
   |
   v
Re-render: code = '123456', isComplete = true
   hint "Submitting automatically..." appears
   |
   v
useEffect([code, isComplete, disabled, ...])
   setTimeout(() => onSubmit('123456'), 2000)   (cleanup clears it)
   |
   v  2 s later
App.handleSubmit('123456')
   setStatus('verifying') -> OtpContainer gets disabled=true
   -> effect cleanup runs, no second timer
   await verifyOtp('123456')
   |
   +-- resolves -> status 'success' -> success message replaces the form
   `-- rejects  -> status 'error', message, attempt + 1
                   -> OtpContainer remounts (key changed): empty boxes, box 1 focused
```

---

7. Important technical challenges

Challenge 1: Typing into a box that already has a digit

Problem: Box 2 holds `5`. The user clicks it and types `7`. The input's value becomes `57` (or `75` if the caret was at the start). The naive `value.slice(-1)` picks `5` in the `75` case, so the user's key is lost. Typing a letter next to the digit (`5a`) must be ignored, and SMS autofill delivers the whole code (`123456`) in one change event.

Solution: Select the box's content on focus (so typing usually replaces it), and normalise every change with one pure helper:

- not all digits -> ignore (`null`);
- exactly 2 digits and the box had a digit -> remove the old digit once, keep the new one;
- otherwise (`''`, one digit, or a whole code) -> use as is.

```
export function newDigits(raw, oldDigit) {
  if (!/^\d*$/.test(raw)) return null
  if (raw.length === 2 && oldDigit) return raw.replace(oldDigit, '')
  return raw
}
// in the input:
onFocus={(e) => e.target.select()}
```

`/^\d*$/` is stricter than `isNaN(value)`: `isNaN(' ')` and `isNaN('1e3')` are both `false`, so `isNaN` lets a space and exponent notation through.

Challenge 2: Paste and autofill fill many boxes at once

Problem: The user pastes `123-456` into box 1. Without a handler the browser drops the whole string into one box. Pasting `99` into box 5 of 6 must not write past the last box.

Solution: Handle `onPaste`, block the default, strip non-digits, then reuse one pure function that writes digits from an index and returns where focus should go. Autofill goes through `onChange` and ends up in the same `fillFrom`.

```
export function fillFrom(otp, index, digits) {
  const next = [...otp]
  let i = index
  for (const d of digits) {
    if (i >= next.length) break
    next[i++] = d
  }
  return { otp: next, focus: Math.min(i, next.length - 1) }
}

const handlePaste = (index, e) => {
  e.preventDefault()
  const digits = e.clipboardData.getData('text').replace(/\D/g, '')
  if (digits) write(index, digits)
}
```

Challenge 3: Focus management with refs

Problem: Focus must move on type, Backspace, arrows, paste, failed submit and remount. Keeping a `focusedIndex` state would re-render on every focus change and can drift from the real DOM focus.

Solution: An array of refs filled by callback refs, and one clamped helper. Backspace is handled in `onKeyDown` because an empty box does not fire `onChange`.

```
const inputRefs = useRef([])
const focusBox = (i) => inputRefs.current[Math.max(0, Math.min(i, length - 1))]?.focus()

<OtpInput ref={(el) => { inputRefs.current[index] = el }} ... />

if (e.key === 'Backspace') {
  e.preventDefault()
  if (otp[index]) return setDigit(index, '')
  if (index > 0) { setDigit(index - 1, ''); focusBox(index - 1) }
}
```

The callback ref uses a block body on purpose: in React 19 a ref callback may return a cleanup function, so the arrow shortcut `(el) => (refs[i] = el)` returns the element and is flagged.

Challenge 4: Auto-submit without stale values or double submits

Problem: A plain `setTimeout` inside `handleChange` (a) is never cancelled, so editing a digit during the delay still submits the old code, (b) can fire twice if the user also clicks Verify, and (c) closes over a stale `otp` value.

Solution: Drive the timer from an effect keyed on the derived `code`. Any edit changes `code` and the cleanup clears the old timer. When `App` sets `status='verifying'`, `disabled` becomes `true`, the cleanup runs and no timer is scheduled. `App` passes a `useCallback`-stable `onSubmit` so the effect does not restart on unrelated renders.

```
useEffect(() => {
  if (!isComplete || disabled) return
  const timer = setTimeout(() => onSubmit(code), autoSubmitDelay)
  return () => clearTimeout(timer)
}, [code, isComplete, disabled, autoSubmitDelay, onSubmit])
```

React StrictMode runs the effect twice in development; the cleanup clears the first timer, so it still submits once.

---

8. Implement in the correct order

#	Step	Working result
1	`useState(emptyOtp(length))`, map to `<input>`s with `value`	N boxes render
2	`handleChange` with `newDigits` (digits only, replace old digit)	Typing works, letters ignored
3	`inputRefs` + `focusBox`, auto-focus on mount, move to next box	Can type a whole code
4	`onKeyDown`: Backspace (clear / go back), ArrowLeft / ArrowRight	Keyboard editing works
5	`onFocus` select	Clicking a box and typing replaces its digit
6	`fillFrom` + `onPaste`	Paste fills boxes

#	Step	Working result
7	`autoComplete="one-time-code"`, `inputMode="numeric"`; autofill via `onChange`	Mobile SMS autofill
8	`<form onSubmit>`, `showErrors`, red border on empty boxes	Validation
9	Auto-submit effect with cleanup	Submits 2 s after the last digit
10	Mock `verifyOtp`, `App` status, `disabled` while verifying	Verify flow
11	Error: bump `key` to clear and refocus; success screen	Retry flow
12	Extract `OtpInput`, ARIA labels, mobile CSS	Clean, accessible, responsive

9. Cover all states and edge cases

State / edge case	Behavior
Initial	Empty boxes, box 1 focused, no errors
Partially filled	Normal borders until the user tries to submit
Submit while incomplete	Red border on empty boxes, alert text, focus first empty box
Fixing a red box	Its red border disappears immediately (derived)
Complete, waiting	Hint "Submitting automatically..."; timer running
Edit during wait	Timer cancelled and restarted only when complete again
Verifying	All inputs + button disabled, "Verifying..."
Success	Form replaced by the success message
Error (wrong code / network)	Error message, boxes cleared, box 1 focused
Letter / space / symbol typed	Ignored
Same digit typed over itself	Box keeps it, focus moves on
Paste longer than the remaining boxes	Extra digits dropped
Paste with no digits	No change
Paste into the middle box	Fills from that box onward
Backspace on empty first box	No-op
Arrow past either end	Focus clamped
Leading zero (`012345`)	Kept: digits are strings
`length` prop changes	Remount with `key={length}` (state is initialised once)
StrictMode double effects	Cleanup keeps one focus call and one timer

10. Final structure

```
_24_otp/
  index.html
  package.json
  src/
    main.jsx
    App.jsx                    # status, mock verify, reset via key
    App.css
    api/
      verifyOtp.js             # mock API
    components/
      OtpContainer.jsx         # state, refs, keyboard, paste, validation, auto-submit
      OtpInput.jsx             # one digit box
    utils/
      otp.js                   # emptyOtp, newDigits, fillFrom (pure)
```

---

11. Full working code

src/utils/otp.js

```
export const emptyOtp = (length) => Array(length).fill('')

// Returns the digits the user just entered into one box, or null to ignore the input.
export function newDigits(raw, oldDigit) {
  if (!/^\d*$/.test(raw)) return null // letters, spaces, "1e3", "-"
  if (raw.length === 2 && oldDigit) return raw.replace(oldDigit, '') // typed next to the old digit
  return raw // '', one digit, or a whole autofilled code
}

// Writes digits into the boxes starting at `index`; returns the new array and the box to focus.
export function fillFrom(otp, index, digits) {
  const next = [...otp]
  let i = index
  for (const d of digits) {
    if (i >= next.length) break
    next[i++] = d
  }
  return { otp: next, focus: Math.min(i, next.length - 1) }
}
```

src/api/verifyOtp.js

```
// Mock API: accepts 123456 after a short delay, rejects everything else.
export function verifyOtp(code) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (code === '123456') resolve({ ok: true })
      else reject(new Error('Invalid code. Please try again.'))
    }, 800)
  })
}
```

src/components/OtpInput.jsx

```
// One digit box. React 19 passes `ref` as a normal prop, so no forwardRef needed.
const OtpInput = ({ ref, index, length, value, invalid, disabled, onChange, onKeyDown, onPaste }) => (
  <input
    ref={ref}
    className={`otp-box${invalid ? ' otp-box--invalid' : ''}`}
    type="text"
    inputMode="numeric"
    autoComplete={index === 0 ? 'one-time-code' : 'off'}
    aria-label={`Digit ${index + 1} of ${length}`}
    aria-invalid={invalid}
    value={value}
    disabled={disabled}
    onFocus={(e) => e.target.select()}
    onChange={onChange}
    onKeyDown={onKeyDown}
    onPaste={onPaste}
  />
)

export default OtpInput
```

src/components/OtpContainer.jsx

```
import { useEffect, useRef, useState } from 'react'
import OtpInput from './OtpInput'
import { emptyOtp, fillFrom, newDigits } from '../utils/otp'

const OtpContainer = ({ length = 6, autoSubmitDelay = 2000, disabled = false, onSubmit }) => {
  const [otp, setOtp] = useState(() => emptyOtp(length))
  const [showErrors, setShowErrors] = useState(false)
  const inputRefs = useRef([])

  const code = otp.join('')
  const isComplete = code.length === length

  const focusBox = (i) => inputRefs.current[Math.max(0, Math.min(i, length - 1))]?.focus()

  useEffect(() => {
    inputRefs.current[0]?.focus()
  }, [])

  // Auto-submit. Editing during the delay, or a manual submit disabling us, clears the timer.
  useEffect(() => {
    if (!isComplete || disabled) return
    const timer = setTimeout(() => onSubmit(code), autoSubmitDelay)
    return () => clearTimeout(timer)
  }, [code, isComplete, disabled, autoSubmitDelay, onSubmit])

  const setDigit = (index, digit) =>
    setOtp((prev) => prev.map((d, i) => (i === index ? digit : d)))

  const write = (index, digits) => {
    const { otp: next, focus } = fillFrom(otp, index, digits)
    setOtp(next)
    focusBox(focus)
  }

  const handleChange = (index, e) => {
    const digits = newDigits(e.target.value, otp[index])
    if (digits === null) return
    if (digits === '') setDigit(index, '')
    else write(index, digits)
  }

  const handlePaste = (index, e) => {
    e.preventDefault()
    const digits = e.clipboardData.getData('text').replace(/\D/g, '')
    if (digits) write(index, digits)
  }
```

```
  const handleKeyDown = (index, e) => {
    if (e.key === 'Backspace') {
      e.preventDefault()
      if (otp[index]) return setDigit(index, '')
      if (index > 0) {
        setDigit(index - 1, '')
        focusBox(index - 1)
      }
    } else if (e.key === 'ArrowLeft') {
      e.preventDefault()
      focusBox(index - 1)
    } else if (e.key === 'ArrowRight') {
      e.preventDefault()
      focusBox(index + 1)
    }
  }

  const handleSubmit = (e) => {
    e.preventDefault()
    if (!isComplete) {
      setShowErrors(true)
      focusBox(otp.indexOf(''))
      return
    }
    onSubmit(code)
  }

  return (
    <form className="otp" onSubmit={handleSubmit} noValidate>
      <div className="otp__boxes" role="group" aria-label={`Enter the ${length}-digit code`}>
        {otp.map((digit, index) => (
          <OtpInput
            key={index}
            ref={(el) => {
              inputRefs.current[index] = el
            }}
            index={index}
            length={length}
            value={digit}
            invalid={showErrors && !digit}
            disabled={disabled}
            onChange={(e) => handleChange(index, e)}
            onKeyDown={(e) => handleKeyDown(index, e)}
            onPaste={(e) => handlePaste(index, e)}
          />
        ))}
      </div>
```

```
      {showErrors && !isComplete && (
        <p className="otp__error" role="alert">Enter all {length} digits</p>
      )}
      {isComplete && !disabled && <p className="otp__hint">Submitting automatically…</p>}
      <button type="submit" disabled={disabled}>
        {disabled ? 'Verifying…' : 'Verify'}
      </button>
    </form>
  )
}

export default OtpContainer
```

src/App.jsx

```
import { useCallback, useState } from 'react'
import './App.css'
import OtpContainer from './components/OtpContainer'
import { verifyOtp } from './api/verifyOtp'

const OTP_LENGTH = 6

const App = () => {
  const [status, setStatus] = useState('idle') // idle | verifying | success | error
  const [message, setMessage] = useState('')
  const [attempt, setAttempt] = useState(0) // bumping it remounts (clears) the boxes

  // Stable identity so the auto-submit effect does not restart on every App render.
  const handleSubmit = useCallback(async (code) => {
    setStatus('verifying')
    setMessage('')
    try {
      await verifyOtp(code)
      setStatus('success')
      setMessage('Phone number verified!')
    } catch (err) {
      setStatus('error')
      setMessage(err.message)
      setAttempt((a) => a + 1)
    }
  }, [])

  return (
    <main className="app">
      <section className="card">
        <h1>Verify your phone</h1>
        <p className="card__sub">Enter the {OTP_LENGTH}-digit code we sent you (demo code: 123456).</p>
        {status === 'success' ? (
          <p className="status status--success" role="status">{message}</p>
        ) : (
          <>
            <OtpContainer
              key={attempt}
              length={OTP_LENGTH}
              disabled={status === 'verifying'}
              onSubmit={handleSubmit}
            />
            {status === 'error' && <p className="status status--error" role="alert">{message}</p>}
          </>
        )}
      </section>
    </main>
```

```
  )
}

export default App
```

src/App.css

```
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: system-ui, sans-serif; background: #eef1f5; color: #1f2328; }

.app { min-height: 100vh; display: grid; place-items: center; padding: 16px; }
.card { width: 100%; max-width: 440px; padding: 32px 24px; background: #fff; border-radius: 12px; box-shadow: 0 4px 20px rgb(0 0 0 / 0.08); text-align: center; }
.card h1 { font-size: 1.4rem; margin-bottom: 6px; }
.card__sub { color: #57606a; font-size: 0.9rem; margin-bottom: 20px; }

.otp { display: flex; flex-direction: column; align-items: center; gap: 12px; }
.otp__boxes { display: flex; gap: 8px; }
.otp-box { width: 48px; height: 56px; font-size: 1.5rem; text-align: center; border: 2px solid #d0d7de; border-radius: 8px; outline: none; }
.otp-box:focus { border-color: #0969da; box-shadow: 0 0 0 3px rgb(9 105 218 / 0.25); }
.otp-box--invalid { border-color: #cf222e; }
.otp-box:disabled { background: #f6f8fa; }
.otp__error, .status--error { color: #cf222e; font-size: 0.85rem; }
.otp__hint { color: #57606a; font-size: 0.85rem; }
.otp button { padding: 10px 28px; border: none; border-radius: 8px; background: #0969da; color: #fff; font-size: 1rem; cursor: pointer; }
.otp button:disabled { opacity: 0.6; cursor: not-allowed; }
.status { margin-top: 12px; }
.status--success { color: #1a7f37; font-weight: 600; }

@media (max-width: 400px) {
  .otp__boxes { gap: 6px; }
  .otp-box { width: 40px; height: 48px; font-size: 1.25rem; }
}
```

src/main.jsx (unchanged from the Vite template)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Quick self-check of the pure logic (`node check.mjs`)

```
import assert from 'node:assert/strict'
import { emptyOtp, newDigits, fillFrom } from './src/utils/otp.js'

assert.equal(newDigits('a', ''), null)
assert.equal(newDigits(' ', ''), null)
assert.equal(newDigits('57', '5'), '7')
assert.equal(newDigits('75', '5'), '7')
assert.equal(newDigits('55', '5'), '5')
assert.equal(newDigits('123456', ''), '123456')
assert.deepEqual(fillFrom(emptyOtp(4), 2, '9876'), { otp: ['', '', '9', '8'], focus: 3 })
console.log('self-check ok')
```

---

12. Wrap-up

Edge cases handled

- Digits only (stricter than `isNaN`), replace instead of append, same digit retyped.
- Backspace on empty box goes back; arrows clamp at the ends.
- Paste with separators, paste longer than remaining boxes, paste with no digits, paste in the middle.
- SMS autofill through `autocomplete="one-time-code"`.
- Red border only on the missing boxes, cleared per box as they are filled.
- Auto-submit cancelled by edits, no double submit with the button, StrictMode safe.
- Wrong code clears and refocuses; leading zeros preserved.

Trade-offs to mention

- **N inputs vs one hidden input:** N inputs are easy to style and reason about, but screen readers announce N fields and some mobile keyboards' Backspace on an empty box sends no key event. A single transparent `<input maxLength={6}>` under N styled cells avoids both, at the cost of trickier caret styling.
- **State inside `OtpContainer`:** simple and self-contained; a parent that needs the live value would need an `onChange` prop or a controlled version.

- **Reset by `key`:** one line, resets state and auto-focus together, but remounts the inputs (fine for 6 boxes).
- **Paste fills from the focused box** rather than always from box 1: more flexible; the other rule is a one-line change (`write(0, digits)`).
- **Auto-submit + button:** fast for users, but a 2 s delay is a guess; some products submit instantly.

What to add with more time

- Tests with React Testing Library + `user-event`: type, Backspace, paste, auto-submit with fake timers.
- Resend button with a countdown (`useEffect` + `setInterval`), attempts-left message, lockout after N failures.
- `mask` prop (`type="password"`), alphanumeric mode (`pattern` prop instead of the hard-coded regex).
- WebOTP API (`navigator.credentials.get({ otp: { transport: ['sms'] } })`) for Android Chrome auto-read.
- Controlled mode (`value` / `onChange` props) for form libraries.
- `AbortController` to cancel a verify request when the component unmounts.

---

13. Counter questions the interviewer may ask

Q1. Why an array of strings and not a single string for the OTP?

Each box needs its own slot, including empty ones in the middle (`['1','','3']`). With a string, a gap shifts every later digit. Strings (not numbers) keep leading zeros and avoid `0` being falsy. The final code is just `otp.join('')`.

Q2. Why is `isComplete` derived instead of stored in state?

It is fully determined by `otp`. Storing it means updating two pieces of state in every handler and risking them disagreeing. Derived values are recomputed each render for free here (6 items).

Q3. Why `useRef` for the inputs and not `document.querySelectorAll`?

Refs are scoped to this component instance; a query selector would grab boxes from a second OTP widget on the same page and breaks with SSR or shadow DOM. Refs also survive re-renders without re-querying.

Q4. Why not keep `focusedIndex` in state?

Focus already lives in the DOM. A state copy re-renders on every focus change and can drift (the user clicks a box, tabs away, the browser autofills). We only need to *set* focus, which `ref.focus()` does.

Q5. Why `/^\d*$/` instead of `isNaN(value)`?

`isNaN(' ')`, `isNaN('')` and `isNaN('1e3')` are all `false`, so `isNaN` lets a space, an empty string and exponent notation through. The regex accepts only the characters 0-9.

Q6. Why not `value.slice(-1)` to take the newest digit?

If the caret is before the old digit, typing `7` into `5` gives `75`, and `slice(-1)` keeps the old `5`. We select the text on focus and, for the two-digit case, remove the old digit instead, which works wherever the caret is.

Q7. Why handle Backspace in `onKeyDown` instead of `onChange`?

On an empty box, Backspace does not change the value, so `onChange` never fires and we cannot move back. `keydown` always fires. We still keep the `onChange` path for clearing, because some Android keyboards send `key: 'Unidentified'` instead of `Backspace`.

Q8. Why `type="text"` with `inputMode="numeric"` and not `type="number"`?

`type="number"` shows spinner arrows, accepts `e`, `+`, `-`, changes value on mouse-wheel, and returns `''` for invalid input so you cannot see what was typed. `inputMode="numeric"` gives the numeric keypad on mobile while keeping full control of the value.

Q9. What if the user pastes the code into the third box?

It fills from box 3 onward and drops what does not fit. If the product wants "paste always fills the whole code", change `write(index, digits)` to `write(0, digits)` in `handlePaste`: one line, which is why the logic lives in `fillFrom`.

Q10. How does SMS autofill work, and how do you support it?

iOS Safari and Chrome on Android offer the code above the keyboard when an input has `autocomplete="one-time-code"`. Accepting it inserts the whole code into the focused box, which arrives as one `onChange` with `'123456'`. `newDigits` passes it through and `fillFrom` spreads it. For fully automatic reading on Android, add the WebOTP API.

Q11. Why put the auto-submit timer in `useEffect` and not in `handleChange`?

A timer started in the handler is never cleared: if the user edits a digit within 2 s, the old code is still submitted, and clicking Verify too submits twice. In an effect keyed on `code`, every change runs the cleanup first, and `disabled` turning `true` cancels it.

Q12. Why is `onSubmit` wrapped in `useCallback` in `App`?

It is in the effect's dependency list. A new function on every `App` render (e.g. when `message` changes) would restart the 2 s timer. `useCallback` with `[]` keeps it stable; it only uses state setters, which are stable already.

Q13. What happens in React StrictMode?

In development, effects run, clean up and run again. The mount focus runs twice (harmless). The auto-submit effect schedules a timer, clears it in cleanup, then schedules one again, so only one submit happens. This is exactly why cleanup matters.

Q14. How do you clear the boxes after a wrong code?

`App` bumps `attempt`, used as `key` on `OtpContainer`. A new key unmounts and remounts it: fresh `otp`, fresh `showErrors`, and the mount effect focuses box 1. The alternative is exposing a `reset()` via `useImperativeHandle`, which is more code for the same result.

Q15. Why `key={index}` when lists usually need stable ids?

Index keys are a problem when items are inserted, removed or reordered, because state sticks to the wrong item. The boxes are a fixed-size positional list that never reorders: box 3 is always box 3. The index is the stable id here.

Q16. What if the user double-clicks Verify or the API is slow?

As soon as `onSubmit` runs, `App` sets `status='verifying'`, which disables the inputs and button and cancels the timer. For a slow API, show a spinner, keep the button disabled, add a timeout with `AbortController`, and show "Network error, try again" on failure.

Q17. What if `length` changes from 6 to 4 at runtime?

`useState(() => emptyOtp(length))` only runs once, so the array stays 6 long. The simplest fix is `key={length}` on the parent's `<OtpContainer>`, which remounts it. Syncing state with an effect is possible but adds a render with the wrong length.

Q18. How is this accessible to screen-reader users?

The group has `role="group"` and an `aria-label` ("Enter the 6-digit code"); each box is labelled "Digit 3 of 6" and gets `aria-invalid` when red. Errors use `role="alert"` so they are announced. All actions work by keyboard and focus is always visible. Red is not the only signal: there is also text.

Q19. Why not a single hidden input with 6 styled boxes on top?

It is a valid alternative with real advantages: one field for screen readers, native Backspace and paste, autofill always lands in the right place. The cost is building a fake caret and selection highlighting. I would mention it as the production-grade option and build N inputs in the interview because each piece is visible and easy to explain.

Q20. Is it safe to verify the OTP on the client?

No. The client only collects the code. The server generates it, stores a hash with an expiry, verifies it, limits attempts and rate-limits resends. The mock checking `123456` in the browser is only a stand-in for that API.

Q21. What if this component is rendered 1,000 times on a page (e.g. a list of forms)?

Each instance has 6 inputs and no global listeners, so it scales linearly and cheaply. Refs and state are per instance, so instances never interfere. The only real concern would be auto-focus: only one instance should take focus on mount, so make `autoFocus` a prop.

Q22. How would you test it?

React Testing Library + `@testing-library/user-event`: type `123456` and assert each box and focus; `{Backspace}` on empty box; `user.paste('12-34-56')`; `vi.useFakeTimers()` and advance 2 s to assert `onSubmit` was called once with `'123456'`; edit during the delay and assert it was not called. The pure helpers are tested with plain asserts.